

Guia de Exercícios: Criando Testes Unitários com `unittest`

Em todos os exercícios, ao escrever sua classe de teste, você deve obrigatoriamente usar comentários para identificar as três etapas do padrão AAA em cada método:

- Arrange (Organizar): Preparar o dado de entrada e o valor esperado.
- Act (Agir): Executar a função a ser testada.
- Assert (Afirmar): Verificar se o resultado é igual ao esperado usando os métodos do `self`.

Exercício 1: O Verificador de Idade

Objetivo: Praticar a estrutura básica e `assertTrue` / `assertFalse` / `assertEqual`.

- Tarefa: Crie uma função `pode_dirigir(idade)` que retorna `True` se a idade for maior ou igual a 18, e `False` caso contrário.
- Desafio: Escreva uma classe de teste com dois métodos:
 1. Um método para testar um caso positivo (ex: 20 anos).
 2. Um método para testar um caso negativo (ex: 16 anos).

Exercício 2: Calculadora de Descontos

Objetivo: Praticar lógica de negócio e múltiplos métodos de teste.

- Tarefa: Crie uma função `calcular_desconto(valor, percentual)` que retorna o valor final do produto com desconto aplicado.
- **Regra de negócio:** Se o percentual informado for maior que 50%, a função deve limitar o desconto a 50%.

Desafio: Crie testes para:

1. Um desconto comum (ex: 10% de R\$ 100).
2. O limite de segurança (ex: tentar aplicar 70% de desconto e verificar se o valor final reflete apenas 50%).
3. Um valor de produto igual a zero.

Exercício 3: Sistema de Cadastro de Usuários

Objetivo: Testar validações e exceções com `assertRaises`.

- Tarefa: Crie uma função `cadastrar_senha(senha)`.
- A senha deve ter pelo menos 8 caracteres.

- Se a senha for menor que isso, a função deve levantar um `ValueError` com a mensagem "`Senha muito curta`".

Desafio: Crie testes para:

1. Teste de Sucesso: Verifique se uma senha válida (ex: 10 caracteres) é cadastrada sem levantar erros.
2. Teste de Exceção: Use `with self.assertRaises(ValueError):` para garantir que tentar cadastrar a senha "`123`" levanta o erro esperado.
3. Desafio Extra: Capture a exceção gerada e verifique se a mensagem do erro é exatamente "`Senha muito curta`".

Exercício 4: Gerenciador de Carrinho de Compras

Objetivo: Testar listas, inserção, remoção e busca usando `assertIn` e `assertNotIn`.

- Tarefa: Crie uma classe simples `CarrinhoDeCompras` que possui uma lista `itens` vazia ao ser inicializada e três métodos:
- `adicionar_item(item)`: adiciona o nome de um produto na lista.
- `remover_item(item)`: remove o produto se ele estiver presente (se não estiver, levanta um `ValueError` com a mensagem "`Item não encontrado`").
- `listar_itens()`: retorna a lista atual de itens.

Desafio: Crie uma classe de teste para verificar:

1. Adicionar um item e verificar se ele está presente na lista (`assertIn`).
2. Remover um item e verificar se ele não está mais na lista (`assertNotIn`).
3. Tentar remover um item que não está no carrinho e verificar se um `ValueError` é disparado.